



Change Requests - CodeSmile

Autore	Matricola
Daniele Pio Scaparra	NF22500101
Grazia Bonagura	NF22500105
Antonio Nappi	NF22500200

Indice

1. Introduzione.....	1
2. CR1 — Miglioramento del Detector “In-Place APIs Misused”.....	2
3. CR2 — Creazione del Call Graph del Progetto.....	2
4. CR3 — Analisi e Visualizzazione del Call Graph.....	3

1. Introduzione

Il presente documento descrive le **Change Request (CR)** identificate per il sistema **CodeSmile**, a seguito dell'analisi dello stato corrente del software e delle sue limitazioni funzionali e architetturali.

Le Change Request costituiscono interventi pianificati di manutenzione, classificabili come correttiva, perfezionativa ed evolutiva, e sono finalizzate a migliorare la qualità delle analisi statiche, ad estendere le funzionalità offerte dal sistema e ad aumentare la comprensibilità dei risultati prodotti.

Le CR qui documentate riguardano:

- il miglioramento di un detector esistente per la rilevazione di specifici code smell;
- l'introduzione di una rappresentazione esplicita delle relazioni di chiamata tra funzioni e metodi del progetto analizzato;

- l'analisi e la visualizzazione strutturata del call graph generato.

Lo scopo di questo documento è definire in modo univoco e non ambiguo il contenuto, gli obiettivi e i benefici attesi di ciascuna Change Request, fornendo una base di riferimento per le successive attività di progettazione, implementazione e verifica.

In accordo con le buone pratiche di ingegneria del software e con gli standard di testing, la descrizione delle Change Request è mantenuta separata dalle attività di definizione della strategia di test, dalla progettazione dei casi di test e dall'analisi degli incidenti di test, che sono trattate in documenti distinti.

2. CR1 — Miglioramento del Detector “In-Place APIs Misused”

Tipologia di manutenzione:

Manutenzione correttiva e perfettiva.

Descrizione del problema

Il detector attuale per lo smell *In-Place APIs Misused* presenta limitazioni che ne riducono l'efficacia complessiva. In particolare:

- la copertura delle API Pandas è parziale;
- non è presente alcun supporto per le API NumPy;
- i risultati di operazioni in-place non utilizzati non vengono sempre rilevati;
- sono presenti falsi positivi in contesti corretti, come assegnazioni esplicite o funzioni wrapper.

Obiettivi

- Ampliare la copertura delle API Pandas rilevanti.
- Introdurre il supporto alle API NumPy.
- Migliorare il riconoscimento dei risultati di operazioni non utilizzati.
- Ridurre la frequenza di falsi positivi e falsi negativi.

Benefici attesi

- Aumento della precisione e del recall del detector.
- Riduzione della rumorosità nei report finali.
- Risultati più coerenti con la semantica delle API di Data Science e Machine Learning.

3. CR2 — Creazione del Call Graph del Progetto

Tipologia di manutenzione:

Manutenzione evolutiva.

Descrizione del problema

Attualmente CodeSmile analizza le funzioni in modo isolato, senza modellare le relazioni di chiamata tra di esse. L'assenza di un call graph limita la comprensione dell'architettura del progetto e il contesto in cui gli smell vengono rilevati.

Obiettivi

- Estrarre tutte le chiamate tra funzioni e metodi all'interno del progetto.
- Costruire automaticamente il call graph del codice analizzato.
- Annotare ciascun nodo del grafo con gli eventuali smell rilevati.
- Esportare il call graph nei formati JSON e DOT.
- Estendere la CLI con un'opzione dedicata alla generazione del call graph.

Benefici attesi

- Introduzione di una visione architeturale del progetto.
 - Migliore comprensione del contesto degli smell rilevati.
 - Base strutturale necessaria per le analisi avanzate previste dalla CR3.
-

4. CR3 — Analisi e Visualizzazione del Call Graph

Tipologia di manutenzione:

Manutenzione evolutiva.

Descrizione del problema

Il call graph generato tramite la CR2 necessita di strumenti dedicati per la sua analisi e visualizzazione. La sola disponibilità di file JSON o DOT non è sufficiente per evidenziare nodi critici, cicli o strutture complesse.

Obiettivi

- Generare una visualizzazione grafica leggibile del call graph (formati PNG/SVG).
- Calcolare metriche di centralità e dipendenza funzionale.
- Identificare automaticamente cicli tra funzioni e metodi.
- Fornire una rappresentazione chiara ed esportabile per l'utente finale.

Benefici attesi

- Migliore interpretabilità dell'architettura del progetto.

- Individuazione immediata di componenti critiche o particolarmente complesse.
- Supporto alle attività di refactoring e manutenzione del codice.

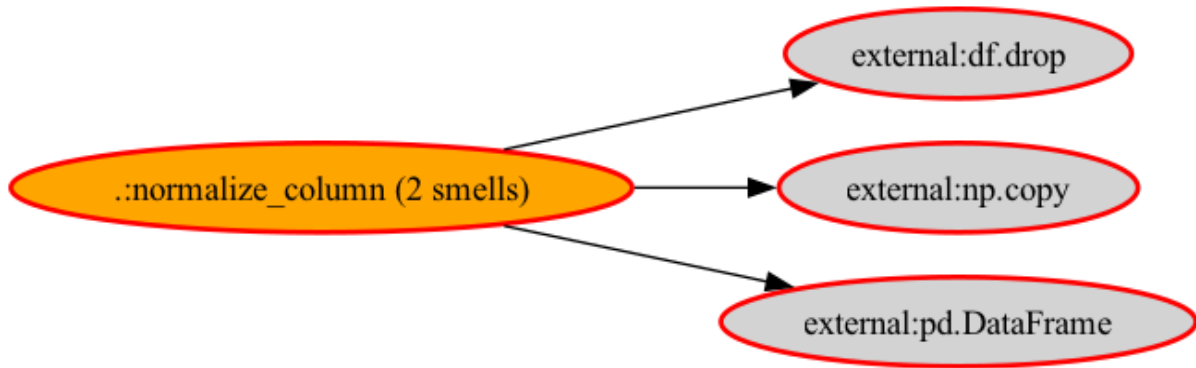


Fig 1. *Esempio di visualizzazione del call graph generato*